

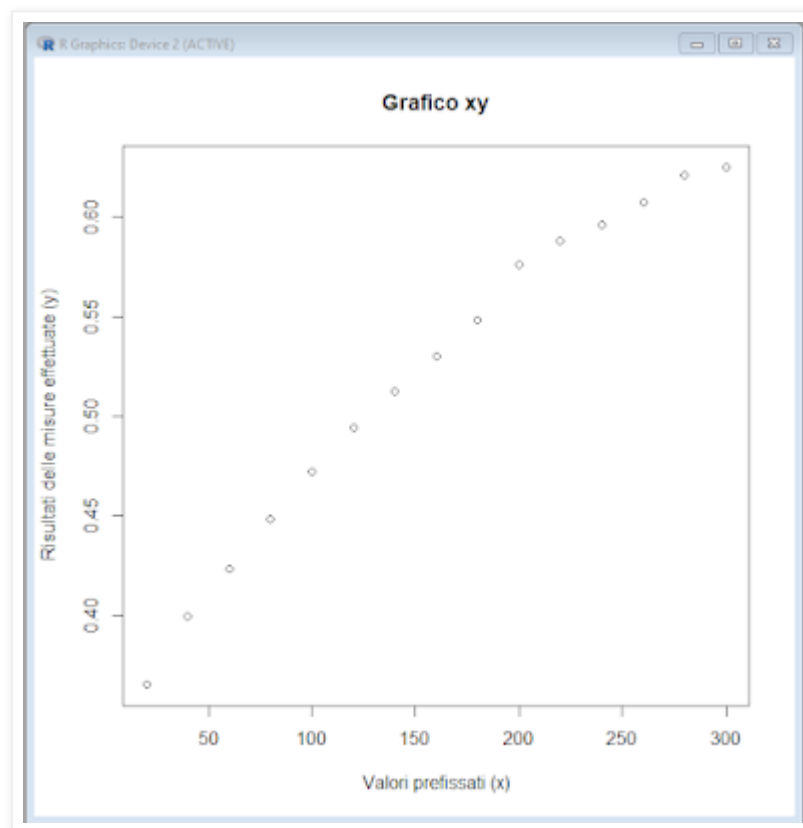
Regressione polinomiale di secondo grado

Supponiamo di avere, per ciascuno di una serie di valori prefissati x (riportati in ascisse), il risultato di altrettante misure y (riportate in ordinate) e riportiamo con la funzione **plot()** i dati sotto forma di un grafico xy .

Copiante il codice che segue nella Console di R e premete ↵ Invio:

```
#  
x <- c(20, 40, 60, 80, 100, 120, 140, 160, 180, 200, 220, 240, 260, 280, 300) # dati in  
ascisse  
y <- c(0.365, 0.399, 0.423, 0.448, 0.472, 0.494, 0.512, 0.530, 0.548, 0.576, 0.588, 0.596,  
0.607, 0.621, 0.625) # dati in ordinate  
plot(y~x, col="black", pch=1, main="Grafico xy", xlab="Valori prefissati (x)",  
ylab="Risultati delle misure effettuate (y)") # traccia il grafico xy  
#
```

Dalla ispezione del grafico risulta evidente una relazione non lineare.



Questo ci fa escludere la possibilità di descrivere la relazione di funzione che intercorre tra le due variabili mediante una retta. E pone il tema di quale sia la curva che potrebbe meglio descriverla.

La prima cosa da valutare è se esiste un modello deterministico di un evento fisico, chimico-fisico, biologico, econometrico o quant'altro, che si assume descriva adeguatamente la relazione di funzione che intercorre tra le due variabili: in tal caso non resta che applicare l'equazione fornita dal modello. In caso contrario è necessario ricorrere ad un modello empirico: ed è questo che consideriamo essere il caso nostro.

Qualora si debba ricorrere ad un modello empirico, la regola fondamentale è impiegare l'equazione più semplice compatibile con una adeguata descrizione dei dati. Nel nostro caso visto che è evidente dall'ispezione del grafico che l'equazione di una retta

$$y = a + b \cdot x$$

calcolata mediante la classica regressione lineare con il metodo dei minimi quadrati sarebbe inadeguata a descrivere la relazione di funzione che lega la y alla x , possiamo pensare di ricorrere ad una equazione appena più complessa, come quella fornita da una **regressione polinomiale di secondo grado** nella forma

$$y = a + b \cdot x + c \cdot x^2$$

e calcolata anch'essa con il metodo dei minimi quadrati [1].

Il rationale di questa scelta è basato sul fatto che i nostri dati descrivono una curva con la concavità rivolta in basso e un raggio di curvatura omogeneo, senza massimi, senza minimi e senza punti di flesso - cioè senza punti di passaggio tra due curvature che vanno in senso opposto - tutti elementi compatibili con il ramo ascendente di una parabola descritta appunto da un polinomio di secondo grado.

Per capire se questa scelta genera una curva che approssima in modo adeguato i nostri dati, disponiamo un breve script per calcolare e rappresentare una regressione polinomiale di secondo grado.

Copiate lo script nella `Console di R` e premete ↵ Invio:

```
# REGRESSIONE POLINOMIALE di secondo grado
#
x <- c(20, 40, 60, 80, 100, 120, 140, 160, 180, 200, 220, 240, 260, 280, 300) # dati in
ascisse
y <- c(0.365, 0.399, 0.423, 0.448, 0.472, 0.494, 0.512, 0.530, 0.548, 0.576, 0.588, 0.596,
0.607, 0.621, 0.625) # dati in ordinate
#
options(scipen=999) # esprime i numeri in formato fisso
#
# calcola la regressione
#
polisec <- lm(y~poly(x, degree=2, raw=TRUE)) # calcola la regressione
summary(polisec) # riporta le statistiche della regressione
#
# traccia il grafico
#
newx <- seq(min(x), max(x), (max(x)-min(x))/1000) # valori x in corrispondenza dei quali
calcolare il valore del polinomio
newy <- predict(polisec, list(x=newx)) # calcola il valore corrispondente del polinomio
plot(y~x, col="black", pch=1, main="Regressione polinomiale di secondo grado",
xlab="Valori prefissati (x)", ylab="Risultati delle misure effettuate (y)") # predispone il
grafico e riporta i dati
lines(newx, newy, col="black", lty=2, lwd=1) # traccia il polinomio
#
options(scipen=0) # ripristina la notazione scientifica
#
```

Le prime due righe di codice servono semplicemente ad assegnare (`<-`) i valori inclusi nella funzione `c()` al vettore `x` e al vettore `y` che in questo modo conterranno i nostri dati.

Poi mediante la funzione `options()` e con l'argomento `scipen=999` facciamo sì che i risultati numerici siano espressi in formato fisso, cosa che li renderà meglio leggibili (questo almeno a mio

modo di vedere: se volete lasciare i risultati nella notazione scientifica eliminate questa riga di codice o, forse meglio, anteponetene a questa riga di codice il simbolo #).

La quarta riga di codice assegna (`<-`) all'oggetto **polisec** mediante la funzione **lm()** i risultati del calcolo della regressione polinomiale **y~poly(x, ...)** di secondo grado (**degree=2**). L'argomento **raw=TRUE** è impiegato perché non ci interessano i polinomi ortogonali (l'opzione di default è **raw=FALSE**).

Se siete interessati ad approfondimenti, ricordo che potete visualizzare ad esempio la struttura dell'oggetto **polisec** digitando **str(polisec)** e potete avere informazioni sulle funzioni impiegate nello script digitando **help(nomedellafunzione)**.

Alla quinta riga di codice la funzione **summary(polisec)** consente di visualizzare le statistiche della regressione polinomiale:

```
Call:
lm(formula = y ~ poly(x, degree = 2, raw = TRUE))
Residuals:
    Min       1Q   Median       3Q      Max
-0.0043421 -0.0023941 -0.0000973  0.0013490  0.0074455
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.3371692308  0.0031879149  105.77 < 0.0000000000000002 ***
poly(x, degree = 2, raw = TRUE)1  0.0015339463  0.0000458427   33.46  0.0000000000000322 ***
poly(x, degree = 2, raw = TRUE)2 -0.0000018851  0.0000001393  -13.53  0.000000012542671 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 0.003579 on 12 degrees of freedom
Multiple R-squared:  0.9985,    Adjusted R-squared:  0.9982
F-statistic: 3879 on 2 and 12 DF, p-value: < 0.00000000000000022
```

La voce **Residuals** riporta le statistiche delle differenze che, dopo l'approssimazione dei punti con il polinomio di secondo grado, residuano tra i punti stessi e la curva calcolata: differenze piccole come quelle qui osservate indicano un'ottima approssimazione della curva ai dati.

La voce **Coefficients** riporta dall'alto verso il basso il valore stimato (**Estimate**) dell'intercetta, del coefficiente di primo grado (**x**) e del coefficiente di secondo grado (**x²**) e ci consente di scrivere l'equazione del polinomio che sarà quindi

$$y = 0.3371692308 + 0.0015339463 \cdot x - 0.0000018851 \cdot x^2$$

Per ciascun coefficiente viene calcolato il valore del *test t di Student* (**t value**) come rapporto tra il valore stimato (**Estimate**) e il suo errore standard (**Std. Error**) e viene infine riportato il valore **p** di probabilità di osservare per caso il valore di **t** riportato: se tale probabilità è sufficientemente bassa ci assumiamo il rischio di considerare significativo il coefficiente in questione. Nel nostro caso essendo i valori di **p** sempre molto piccoli (di gran lunga inferiori al valore **p = 0.05** generalmente considerato come valore soglia per la significatività) possiamo affermare che tutti e tre i coefficienti del polinomio di secondo grado sono significativi e lo rendono un'equazione in grado di approssimare adeguatamente i dati forniti.

Il valore del *coefficiente di correlazione multipla R²* molto elevato (**Multiple R-squared: 0.9985**) da una misura della bontà dell'approssimazione, confermata dalla sua significatività con il *test F* (**p-value: < 0.00000000000000022**).

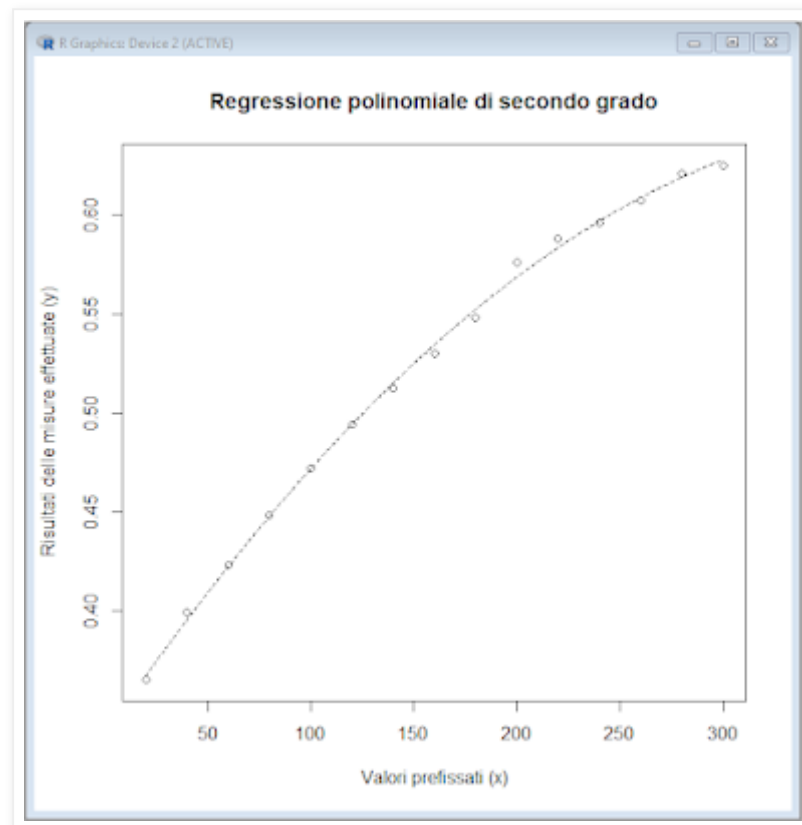
Infine in quattro passaggi viene tracciato il grafico che riporta i punti e traccia il polinomio che li approssima:

→ l'intervallo compreso tra il valore minimo **min(x)** della **x** e il valore massimo **max(x)** della **x** viene suddiviso per ottenere i valori **x** (salvati nell'oggetto **newx**) in corrispondenza dei quali calcolare il valore del polinomio;

→ i valori y del polinomio corrispondenti alle x contenute in **newx** sono calcolati e riportati nell'oggetto **newy**;
→ la funzione **plot()** viene impiegata per riportare sul grafico di dati contenuti in **x** e in **y**;
→ la funzione **lines()** viene impiegata per sovrapporre ai dati il tratto di polinomio calcolato e salvato in **newx** e in **newy**.

Potete riportare i dati modificando il simbolo impiegato (**pch=...**) e il suo colore (**col="..."**), come pure tracciare il polinomio modificando l'aspetto o stile della linea tracciata (**lty=...**), il suo colore (**col="..."**) e il suo spessore (**lwd=...**) [2].

Il grafico - complemento ai test statistici da cui non è possibile prescindere - conferma che la regressione polinomiale di secondo grado fornisce una eccellente approssimazione ai dati.



Da notare che il grafico è stato tracciato esclusivamente all'interno dei dati, quindi in un'ottica di interpolazione: e questo ci ricorda anche che, trattandosi di un modello empirico, è da escludere l'impiego del polinomio di secondo grado per la estrapolazione, cioè per trarre conclusioni al di fuori dei dati impiegati per calcolarlo.

A chiudere lo script è la funzione **options()** che con l'argomento **scipen=0** ripristina l'espressione dei numeri nella notazione scientifica [3].

Lo script può essere riutilizzato semplicemente riportando manualmente in **x** e in **y** i nuovi dati, o meglio ancora importandoli da un file [4].

Se siete interessati al tema trovate in un altro post come - in assenza di un'equazione basata su un modello teorico - situazioni più complesse, ma che si possono riscontrare nella pratica, possono essere affrontate impiegando per approssimare i punti la regressione polinomiale di terzo grado.

[1] Il **metodo dei minimi quadrati** prevede - ed è quello che abbiamo assunto essere il nostro caso - che la variabile in ascisse (x) abbia *valori prefissati*, che la variabile in ordinate (y) sia

rappresentata da *misure* (affette da un errore casuale distribuito normalmente) effettuate corrispondenza delle x e *minimizza* la somma dei quadrati delle differenze $y - y'$ ovvero delle differenze tra ciascun valore misurato y e il valore $y' = f(x)$ corrispondente calcolato mediante la regressione (lineare o polinomiale). Esempi possono essere la risposta y di un sensore misurata in corrispondenza di una serie di concentrazioni date x di una sostanza chimica, la misurazione di un evento y in corrispondenza di una serie data di tempi x , e così via.

[2] Trovate le relative indicazioni:

- nel post [Cambiare i simboli dei punti di R](#)
- nel post [Visualizzare i colori disponibili in R](#)
- nel post [Cambiare gli stili delle linee di R](#).

[3] La notazione scientifica è una notazione esponenziale, cioè una notazione che consente di scrivere un numero N assegnato come prodotto di un opportuno numero a per la potenza di un altro numero b^k essendo b la base della notazione esponenziale. In particolare la notazione scientifica è una notazione esponenziale in base 10 ($b = 10$). Tuttavia "notazione esponenziale" non è sinonimo di "notazione scientifica" in quanto esistono notazioni esponenziali che impiegano una base diversa da quella impiegata dalla notazione scientifica: ne sono un esempio la notazione esponenziale in base 2 ($b = 2$) impiegata in informatica, e la notazione esponenziale in base e ($b = e$) - essendo il numero di Nepero $e = 2.718\ 281\ 828\ 459...$ - impiegata in campo matematico e in campo scientifico.

[Precisazione aggiuntiva: in **R** il separatore delle cifre decimali è il punto (.) e come già riportato altrove questa convenzione per ragioni di omogeneità viene adottata non solo negli script ma anche nei file di dati e in tutto il testo di questo sito, quindi anche nell'espressione del numero e di Nepero. Per le norme che regolano la punteggiatura all'interno dei numeri rimando al post [Come separare i decimali e come raggruppare le cifre](#)].

[4] Alla [pagina Indice](#) nel capitolo **R - gestione dei dati** trovate i collegamenti ai post che contengono gli script per importare i dati da file esterni.
